SPRINT 3 BACKLOG
ShadowStack: Predictive Cloud Cost Optimizer

FastAPI — PostgreSQL — React + Vite — D3.js — PyTorch — Docker

Product Backlog v1.2 · Sprint 3 of 4

Project Team: Aniruddha Bhide (Lead/Backend/ML), Sanay Krishna (Frontend/Docs)
Course: BCA 6th Semester Project **Sprint Duration:** 09 Mar – 05 Apr 2026
Total Planned SP: 28 **Lead:** Sanay Krishna

Sprint 3 Goal

Complete the React + Vite dashboard by connecting all four D3.js chart components to live backend APIs and implementing every required interactive visualisation. Aniruddha delivers the cost history endpoint for frontend consumption. Sanay leads all frontend implementation and owns QA documentation and Agile artefacts (redistributed from Arpita M).

Field	Definition
ID	Unique backlog item identifier (PBI-033 to PBI-046)
User Story	*As a [role], I want [goal], so that [benefit]*
Priority	HIGH = MVP-critical MED = MVP, not blocking
SP	Story Points (Fibonacci: 1, 2, 3, 5, 8, 13)
Owner	Primary responsible team member

Sprint 3 At a Glance

Sprint	Epic / Workstream	Planned SP	Stories	Lead	Deadline
S3	EP-8: Frontend Dashboard & Charts	25	13	Sanay	05 Apr 2026
S3	EP-2/7: Backend Support	3	1	Aniruddha	05 Apr 2026
	Total	**28**	**14**		

† Items PBI-045 and PBI-046 were previously owned by Arpita M. Reassigned to Sanay Krishna effective 08 Mar 2026.

EP-8 — Frontend Dashboard & Charts

Sanay Krishna — Full Dashboard Implementation 25 SP

ID	Title	User Story	Acceptance Criteria	Pri.	SP
PBI-033	Axios API Integration Layer	As a frontend developer, I want a centralised Axios service layer so that all API calls have uniform error handling.	Axios instance with base URL from `VITE_API_BASE_URL`; interceptors for 4xx/5xx; loading and error states per request; typed response models	HIGH	2
PBI-034	Cost Trend Line Chart	As a developer, I want an interactive line chart of historical and predicted costs so that I can identify cost patterns.	D3.js area/line chart; historical series converging into dotted ML predictions; gradient confidence interval; tooltips; 7d/30d/90d time-range selector	HIGH	3
PBI-035	Service Breakdown Donut Chart	As a developer, I want a donut chart breaking costs by service so that I can identify the most expensive components.	$\geq$3 segments; legend with percentages; tooltips; updates on filter change; data from backend `/api/predictions`	HIGH	3
PBI-036	Predicted vs. Actual Bar Chart	As a developer, I want a grouped bar chart comparing predicted and actual costs so that model accuracy is visually obvious.	Grouped bars per time bucket; variance % shown; custom tooltip tracking over/under metrics; colour-coded above/below threshold	HIGH	3
PBI-037	ML Model Metrics Panel	As a system admin, I want a panel showing MAE, RMSE, and R^2 for all models so that I can monitor prediction quality.	Accuracy %, R^2 Score, MAPE, MAE, RMSE displayed as metric cards with animated progress bars; model version selector present	HIGH	2
PBI-038	Time-Range Date Filters	As a user, I want date-range controls on all charts so that I can zoom into specific time periods.	7d/30d/90d buttons update URL params (`?range=30D`); all charts re-fetch on param change; default 30 days; state persists across routes	MED	2
PBI-039	Interactive Filtering & Sorting	As a user, I want to filter cost data by service and sort records so that I can focus on relevant data.	Cloud service dropdown updates URL param (`?service=EC2`); `mockAdapter.ts` mathematically	MED	2

† Reassigned from Arpita M to Sanay Krishna.

EP-2/7 — Backend Support

Aniruddha Bhide — Cost History Endpoint — 3 SP

ID	Title	User Story	Acceptance Criteria	Pri.	SP
PBI-043	Cost History Retrieval Endpoint	As a frontend developer, I want `GET /api/predictions` so that charts display accurate prediction history.	Returns prediction records in descending chronological order; formatted timestamps; consumed by `useDashboardData.ts`; response $<$500 ms	HIGH	2

Unplanned Backend Scope — Identified During Sprint 3

The following backend items were not in the original Sprint 3 backlog but are being delivered in Sprint 3 as they are critical path dependencies for Sprint 4. They are formally tracked here as additional scope.

- **GitHub Webhook HMAC hardening** — `X-Hub-Signature-256` validation; BackgroundTask inference to prevent GitHub timeout
- **Automated PR comment posting** — `post_github_comment` helper using `urllib`; auto-triggered after prediction
- **Dynamic OAuth token system** — `integrations` PostgreSQL table; `POST /api/integrations/github/webhook` endpoint; per-repo token storage replacing global `.env` token
- **Frontend data endpoint** — `GET /api/predictions` with descending chronological ordering

These items are formally documented in the Sprint 3 Completion Report as unplanned scope.

Sprint 3 Success Criteria

#	Criterion	Verified By
1	All four D3.js visualisations render data and respond to filter changes	Sanay
2	Dashboard page load time < 2 seconds	Chrome DevTools
3	30-second polling engine active; `StatusBar` shows live status	Manual test
4	URL filter state (`?range=&service=`) persists across route changes	Manual test
5	Frontend unit test coverage $\geq 70\%$ on Sprint 3 modules (Vitest)	`vitest run --coverage`
6	`GET /api/predictions` returns data; consumed by `useDashboardData.ts`	Postman + browser
7	`mockAdapter` fallback activates when API is unreachable	Manual test (stop backend)
8	GitHub webhook HMAC validation and background task active	Test with simulated payload

Definition of Done (Sprint 3)

A Sprint 3 item is **Done** when:

1. All acceptance criteria verified and signed off by item owner.
2. Code merged to `main` via a reviewed GitHub pull request.
3. Relevant Vitest unit tests written and passing (`vitest run`).
4. No new ESLint errors introduced (`eslint src/`).
5. Feature renders correctly in the running Vite dev server (`localhost:5173`).
6. Impacted documentation updated (README or codebase map).
7. Item marked Done on the Agile project board.

ShadowStack — Sprint 3 Backlog · Product Backlog v1.2 · Mar 9 – Apr 5, 2026
Aniruddha Bhide (CU23BSC001A) · Sanay Krishna (CU23BCA0049A)